

MODULE 03 · DOMAIN 1 - FOUNDATION MODEL INTEGRATION, DATA MANAGEMENT, AND COMPLIANCE

Feeding the Model

Task 1.3. Validating, extracting, enriching and formatting data so a foundation model can actually use it.

DOMAIN 1 of 5 EXAM WEIGHT 31% MODULE 3 of 22

A foundation model cannot tell you its input was bad. It will produce a fluent, confident answer from a truncated table, a mis-encoded character set, or a document that failed to parse.

Five stages between raw data and a prompt

01

Ingest

Batch from S3, or streaming from Kinesis or MSK.

02

Validate

Reject or quarantine what fails the rules.

03

Extract

Get text and structure out of documents, audio, images.

04

Enrich

Entities, normalisation, redaction, metadata.

05

Format

Build the request the model actually expects.

ON THE EXAM

Skills 1.3.1 to 1.3.4 map onto stages two to five in order. When a question describes a data problem, work out which stage it belongs to first.

What quality means when an FM is downstream

- ◆ **Completeness.** A missing field becomes an invented field once the model fills the gap.
- ◆ **Freshness.** Stale records produce answers that are wrong rather than obviously old.
- ◆ **Encoding and structure.** A failed parse often yields plausible-looking rubbish, not an empty result.
- ◆ **Duplication.** Duplicates skew retrieval and waste tokens on repeated context.
- ◆ **Sensitive data.** PII entering a prompt is a governance failure that no downstream control can undo.

ON THE EXAM

The last one has no remedy. Once PII reaches the model, it may be in logs, in caches and in your context history. Filter at ingestion, not at output.

Four ways to validate, and when each fits

TOOL	SHAPE	REACH FOR IT WHEN
AWS Glue Data Quality	Serverless rules in DQDL	Tabular data in the Data Catalog or a Glue job
Custom Lambda	Your own code	Bespoke logic, or a small per-record check
SageMaker Processing	Managed batch compute	Heavy transformation over a large dataset
CloudWatch metrics	Measurement, not gating	Tracking quality over time and alarming on drift

ON THE EXAM

Skill 1.3.1 also names SageMaker Data Wrangler. Its capabilities have largely moved into SageMaker Canvas and Unified Studio, but it remains on the in-scope list.

AWS Glue Data Quality

- ◆ Rules are written in **DQDL**, a small domain-specific language. *IsComplete, IsUnique, ColumnValues, ColumnLength*.
- ◆ **Rule recommendations**. It profiles your data and proposes a starting ruleset, so you are not writing rules from a blank page.
- ◆ **Anomaly detection** uses historical profiles to flag deviations no static rule would catch.
- ◆ Runs against the **Data Catalog** or inside a **Glue ETL job**, so it can gate a pipeline mid-flight.
- ◆ Failures emit **EventBridge** events. That is how you pause a pipeline or page someone.

ON THE EXAM

Serverless, built on the open-source DeeQu framework, and it writes its own first draft of rules. That combination is why it beats hand-rolled validation in exam answers.

An insurer loads a nightly file of 50 million claim records into Amazon S3, catalogues it in AWS Glue, then uses a foundation model to generate claim summaries for adjusters.

Twice this quarter an upstream system truncated a column, and the model produced confident summaries citing amounts that were wrong by a factor of a thousand. Nobody noticed for days because no job failed. The team wants to stop bad batches reaching the model, and wants to be alerted when the shape of the data drifts even if every individual rule still passes.

What should the insurer implement?

- ☐ A Add a Lambda function that samples 1,000 records per batch and asks a foundation model whether the data looks correct
- ☐ B Use AWS Glue Data Quality with a DQDL ruleset and anomaly detection, and stop the pipeline on an EventBridge failure event
- ☐ C Publish record counts to CloudWatch and create an alarm on a sudden drop
- ☐ D Enable S3 Versioning so a bad batch can be rolled back after it is detected

◆ PAUSE AND DECIDE

What should the insurer implement?

- A

 Sample records and ask an FM if they look right
Uses a non-deterministic tool for a deterministic job, on a sample rather than the batch. Truncation is a rule check, not a judgement call.
- B

 Glue Data Quality with DQDL and anomaly detection
Rules catch truncation deterministically, anomaly detection catches the drift no rule was written for, and the EventBridge event is the gate.
- C

 Record counts plus a CloudWatch alarm
Measures the wrong thing. A truncated column does not change the record count, so the alarm never fires.
- D

 S3 Versioning for rollback
Recovery, not prevention. The requirement is to stop bad batches reaching the model, and this lets them through then helps you clean up.

THE SIGNAL

Two requirements, deliberately: a hard gate for known failures, and detection for unknown ones. Only Glue Data Quality gives you both, because rules and anomaly detection answer different halves.

Getting content out of documents, audio and images

SERVICE	INPUT	WHAT IT GIVES YOU
Amazon Textract	Scanned documents, forms	Text, tables, form key-value pairs, layout
Amazon Transcribe	Audio and video	Transcript with speakers and timestamps
Amazon Rekognition	Images and video	Objects, faces, moderation labels, text in images
Amazon Comprehend	Text	Entities, key phrases, sentiment, language, PII
Amazon Comprehend Medical	Clinical text	Medical entities, and PHI detection
Bedrock Data Automation	All four modalities	Structured output from one managed service
A multimodal FM	Images, documents, text	Whatever your prompt asks for

ON THE EXAM

The first five are deterministic, single-purpose and cheap. The last two are generative and flexible. Choosing between those groups is the whole skill.

Bedrock Data Automation

- ◆ One managed service extracting structured insight from **documents, images, audio and video**.
- ◆ **Standard output** gives predefined insights per modality. Transcripts, document semantics, video and scene summaries.
- ◆ **Blueprints** define custom output: your field names, your data types, natural language instructions per field.
- ◆ A **project** groups those output configurations and is what you reference when you invoke it.
- ◆ It can act as a **parser inside a Bedrock Knowledge Base**, which matters a great deal in module 5.

ON THE EXAM

Blueprint instruction optimisation refines your extraction instructions from around ten labelled examples. That is accuracy tuning without training a model.

Bedrock Data Automation or the point services?

USE BDA WHEN

Mixed content, one output shape

- Input spans several modalities at once
- You want one service and one job to manage
- You need fields defined by business meaning
- It will feed a knowledge base

USE POINT SERVICES WHEN

One modality, one known job

- Only forms, or only audio, or only images
- You need Textract's exact table geometry
- Cost per item matters at very high volume
- You already own a working pipeline

THE TIEBREAKER

Chaining Textract, Transcribe and Rekognition together with orchestration is a real answer, and it is more operational overhead than one BDA project.

Extract first, or just send it to a multimodal model?

EXTRACT FIRST

Deterministic, auditable, cheap

- Same input always gives the same output
- Confidence scores you can threshold on
- Far cheaper per page at volume
- You can log exactly what was extracted

SEND THE IMAGE

Flexible, contextual, dearer

- Understands layout and meaning together
- Handles questions you did not anticipate
- No pipeline to build or maintain
- Non-deterministic and priced per token

THE TIEBREAKER

'Consistent, auditable extraction of known fields' means extract first. 'Answer arbitrary questions about the document' means give the model the document.

A property firm receives 12,000 items a day: scanned surveys as PDFs, site photographs, and recorded voice notes from surveyors. All three relate to the same property file and must be searchable together.

For each item the business wants the same six fields extracted, named consistently regardless of source. Engineering has one small team and is explicit that it does not want to build and maintain orchestration across several extraction services. The output will populate a Bedrock knowledge base.

Which approach best meets the requirements?

- A Use Amazon Textract for PDFs, Amazon Rekognition for photographs and Amazon Transcribe for voice notes, orchestrated by AWS Step Functions with Lambda functions mapping each output to the six fields
- B Use Amazon Bedrock Data Automation with a project containing custom blueprints that define the six fields for each modality
- C Send every item directly to a multimodal foundation model with a prompt requesting the six fields
- D Use Amazon Comprehend custom entity recognition after converting all three input types to text

Which approach best meets the requirements?

- A

 Three point services orchestrated by Step Functions
Works, and is exactly the orchestration the team said it does not want. Three services, three output shapes, and mapping code to maintain per modality.
- B

 BDA project with custom blueprints per modality
One service covering all three modalities, blueprints enforce the same six field names across them, and BDA feeds a knowledge base as a parser.
- C

 Multimodal FM with a prompt
No consistency guarantee on field naming across 12,000 items a day, no confidence scores, and the most expensive option per item.
- D

 Comprehend custom entities after conversion to text
Still needs the conversion step this question is about, then adds custom model training. Photographs also lose everything non-textual.

THE SIGNAL

Three modalities, one output schema, a small team refusing orchestration, and a knowledge base destination. Every clause points at BDA, which is unusual and is the signal.

Formatting the request the model expects

- ◆ **Converse** takes a messages array, a separate **system** block and an **inferenceConfig** block.
- ◆ Put instructions in **system**, not in the user turn. They survive conversation trimming.
- ◆ Documents and images are **content blocks**, not text pasted into the prompt.
- ◆ Convert tables to **clear text or JSON**. Raw CSV with 80 columns reads badly.
- ◆ **InvokeModel** takes the provider's native body, which differs per provider.

ON THE EXAM

The guide names JSON for Bedrock requests, structured data for SageMaker endpoints, and conversation formatting for dialogue. Three shapes, three destinations.

Conversation state is your problem, not Bedrock's

- ◆ **Bedrock stores nothing between calls.** Converse is stateless. There is no session on the service side.
- ◆ You send the **entire history** on every turn, and you pay input tokens for all of it, every time.
- ◆ Store history in **DynamoDB** keyed by session, with a TTL to expire it. The exam guide names this pattern.
- ◆ Long conversations need **trimming or summarising** before they exceed the context window.
- ◆ **AgentCore Memory** is the managed alternative, covered in module 7.

ON THE EXAM

Cost questions about a chat application usually turn on this. A conversation that grows unbounded has a bill that grows quadratically, because every turn resends every previous turn.

A support chat application on Amazon Bedrock keeps full conversation history in a Lambda function's memory and sends it on every Converse call. Sessions frequently run past 60 turns. Two problems have emerged. Costs have grown far faster than conversation volume, and long sessions occasionally fail with a validation error about input length. Users must still be able to refer back to details mentioned early in a conversation, and the team wants sessions to survive a Lambda cold start.

What should the team implement?

- ☐ A Store history in Amazon DynamoDB with a session key and TTL, and send a rolling window of recent turns plus a running summary of earlier ones
- ☐ B Increase maxTokens and enable a model with a larger context window
- ☐ C Store history in DynamoDB and send only the most recent five turns on every call
- ☐ D Enable Lambda provisioned concurrency so the in-memory history is not lost

◆ PAUSE AND DECIDE

What should the team implement?

- A

DynamoDB with TTL, rolling window plus running summary
Durable across cold starts, bounds the input so the length error stops, and the summary preserves early detail that a raw window would drop.
- B

Bigger context window and higher maxTokens
maxTokens caps output, not input, so it does not touch this error. A larger window raises the ceiling and makes the cost problem worse.
- C

DynamoDB, last five turns only
Fixes cost and length, and breaks the stated requirement. Turn 60 can no longer refer back to something said at turn 3.
- D

Provisioned concurrency
Addresses only the cold start. Cost and context length are untouched, and in-memory state is still lost on scale-out.

THE SIGNAL

Three requirements again: durability, bounded input, and early recall. C fails only the third, which makes it the strong distractor. Summarisation is what buys all three at once.

What Amazon Comprehend gives you

CAPABILITY	RETURNS	TYPICAL PIPELINE USE
Entity detection	People, places, dates, quantities	Metadata for filtering retrieval
PII detection	Entity types and offsets	Redaction before the prompt is built
Key phrase extraction	Salient noun phrases	Tags and search facets
Sentiment	Positive, negative, neutral, mixed	Routing and prioritisation
Language detection	Language code with a score	Choosing model or prompt per language
Custom classification	Your own labels	Deterministic routing at low cost

ON THE EXAM

Comprehend returns offsets, which is what makes reliable redaction possible. A model asked to redact returns rewritten text you then have to trust.

Comprehend or a foundation model?

USE COMPREHEND

Fixed task, high volume

- The task is one of its six capabilities
- You need the same answer every time
- You need character offsets for redaction
- Volume makes per-item cost matter

USE AN FM

Open-ended or novel task

- The task is not a standard NLP task
- You need reasoning about the content
- Requirements change faster than a model retrain
- Volume is low enough that cost is not the driver

THE TIEBREAKER

A model can do everything Comprehend does, less predictably and at higher cost. Comprehend cannot do what a model does. That asymmetry decides most of these questions.

A telecoms company processes 40 million customer support transcripts a month. Before any transcript reaches a foundation model, all personal information must be removed, and the compliance team must be able to prove which characters were redacted in each transcript.

An engineer has proposed prompting a foundation model to rewrite each transcript with personal details removed, arguing this handles edge cases better than a fixed detector.

How should the redaction step be built?

- A Prompt a foundation model to rewrite each transcript with personal details removed
- B Use Amazon Comprehend PII detection to obtain entity types and offsets, and redact those spans deterministically in a Lambda function
- C Apply an Amazon Bedrock guardrail with sensitive information filters to the transcripts
- D Use Amazon Macie to scan the transcripts in Amazon S3 and quarantine any object containing personal information

◆ PAUSE AND DECIDE

How should the redaction step be built?

A FM rewrites each transcript

Returns new text with no record of what changed, so the audit requirement fails. It is also the most expensive option at 40 million items.

C Bedrock guardrail sensitive information filters

Guardrails act at invocation, so data still reaches Bedrock. The requirement is removal before the model sees it, and guardrails are billed per evaluation.

B Comprehend PII detection, redact by offset in Lambda

Offsets are an auditable record of exactly which spans were replaced, the operation is deterministic, and per-item cost suits this volume.

D Macie scanning S3

Discovers and classifies sensitive data at rest. It reports on objects, it does not redact spans inside a transcript.

THE SIGNAL

'Prove which characters were redacted' is the whole question. Only an offset-based approach produces that evidence. Rewriting destroys it by construction.

Streaming ingestion into a GenAI pipeline

- ◆ **Kinesis Data Streams** or **Amazon MSK** for high-throughput event streams that need ordering and replay.
- ◆ **EventBridge** for event-driven routing between services when volume is modest and the shape varies.
- ◆ **S3 Event Notifications** to trigger extraction the moment a document lands. The commonest pattern by far.
- ◆ **Buffer before you invoke.** Per-event model calls at stream rates will throttle and will cost a fortune.
- ◆ **SQS between stages** gives you retries, dead-letter queues and back-pressure the model call badly needs.

ON THE EXAM

Foundation models are slow and rate-limited compared with a stream. Any design that invokes a model once per stream event needs a buffer between them.

A manufacturer runs a nightly pipeline that reprocesses its entire 4 TB document repository through Amazon Textract and a foundation model, regenerating summaries and refreshing a knowledge base. The job now takes eleven hours and has begun overrunning into the working day.

Analysis shows that roughly 2% of documents change on any given day. The team has proposed increasing SageMaker Processing instance counts to shorten the run, and separately proposed switching to a faster, cheaper model to reduce time per document.

What is the most effective change?

- ☐ A Increase the SageMaker Processing instance count to parallelise the existing full reprocessing job
- ☐ B Switch to a smaller foundation model to reduce per-document processing time
- ☐ C Process only changed documents, detected by S3 Event Notifications or object modification timestamps, and update the knowledge base incrementally
- ☐ D Move the pipeline to Bedrock batch inference and accept a longer overall runtime

What is the most effective change?

- A

 More SageMaker Processing instances
Makes the wrong work finish faster and multiplies the cost of doing it. Ninety-eight per cent of the documents did not change.
- B

 Smaller foundation model
Trades summary quality for speed, on work that should not run at all. It also does nothing about the Textract portion of the eleven hours.
- C

 Incremental processing of changed documents only
Removes 98% of the work rather than accelerating it. Runtime, Textract cost and model cost all fall by roughly the same proportion.
- D

 Bedrock batch inference
Halves the token price and lengthens the runtime, when the problem stated is that the runtime is already too long.

THE SIGNAL

'Roughly 2% of documents change' is the entire question wearing a disguise. Before optimising how fast work runs, ask whether the work needs doing.

The five things to carry forward

- ◆ A model **cannot signal bad input**. Validation moves upstream and becomes explicit.
- ◆ **Glue Data Quality** for tabular rules plus anomaly detection. Rules catch known failures, anomaly detection catches the rest.
- ◆ **BDA** for mixed modalities with one output schema. **Point services** for one modality with a known job.
- ◆ **Comprehend** when the task is fixed, high volume and needs offsets. **An FM** when it needs reasoning.
- ◆ **Bedrock is stateless**. Conversation history is yours to store, trim and pay for on every turn.

ON THE EXAM

Module 4 starts the biggest topic in the domain. Knowledge Bases appeared 76 times in the practice bank, and vector stores are where those questions begin.